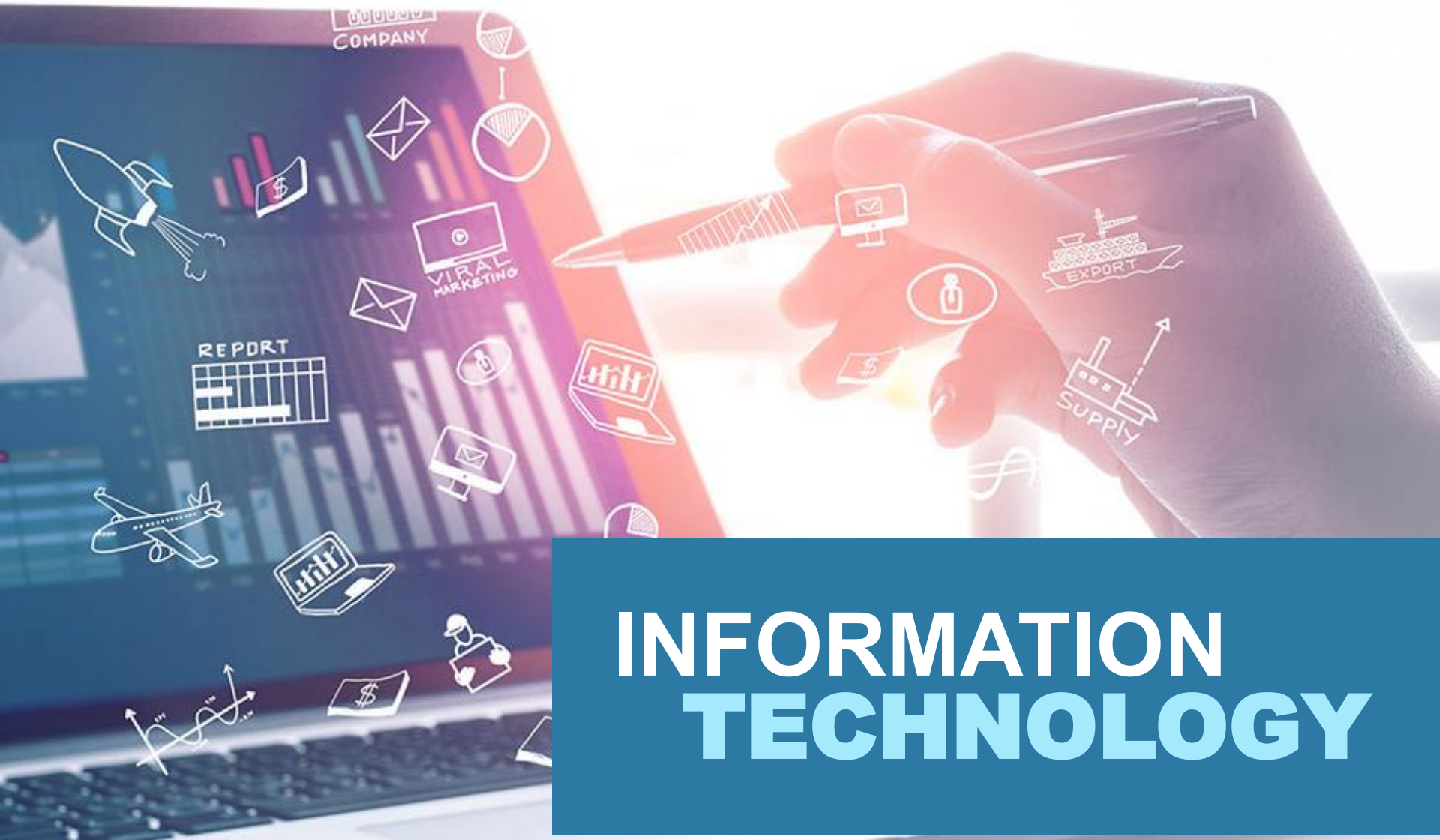
Data Structures and Algorithms



INFORMATION TECHNOLOGY

Data Structures and Algorithms

Rationale

- Computer science is a field of study that solves a variety of problems by using computers.
- To solve a given problem by using computers, you need to design an algorithm for the same.
- Multiple algorithms can be designed to solve a particular problem.
- An algorithm that provides the maximum efficiency should be used for solving the problem.
- The efficiency of an algorithm can be improved by using an appropriate data structure.

Data Structures and Algorithms

Rationale (Contd.)

- Data structures help in creating programs that are simple, reusable, and easy to maintain.
- This module will enable a learner to select and implement an appropriate data structure and algorithm to solve a given programming problem.

Data Structures and Algorithms

Objectives

- In this session, you will learn to:
  - Explore the role of algorithms and data structures in problem solving
  - Design algorithms and measure their efficiency

Data Structures and Algorithms

Explore the Role of Algorithms and Data Structures in Problem Solving

- Problem solving is an essential part of every scientific discipline.
- To solve a given problem by using a computer, you need to write a program for it.
- A program consists of two components, algorithm and data structure.

Role of Algorithms

- The word, algorithm, is derived from the name of the Persian mathematician, Al Khwarizmi.
- An algorithm can be defined as a step-by-step procedure for solving a problem.
- An algorithm helps the user to get the correct result in a finite number of steps.

Data Structures and Algorithms

Role of Algorithms (Contd.)

- An algorithm has five important properties:
  - Finiteness
  - Definiteness
  - Input
  - Output
  - Effectiveness

Role of Algorithms (Contd.)

- A problem can be solved using a computer only if an algorithm can be written for it.
- In addition, algorithms provide the following benefits:
  - Help in writing the corresponding program.
  - Help in dividing difficult problems into a series of small solvable problems.
  - Help in making the process consistent and reliable.
  - Help in making decision.

Data Structures and Algorithms

Role of Data Structures

- Multiple algorithms can be used to solve the same problem.
- Some algorithms may solve the problem more efficiently than the others.
- An algorithm that provides the maximum efficiency should be used to solve a problem.
- One of the basic techniques for improving the efficiency of algorithms is to use an appropriate data structure.
- Data structure is defined as a way of organizing the various data elements in memory with respect to each other.

Data Structures and Algorithms

Role of Data Structures (Contd.)

- Data can be organized in many different ways. Therefore, you can create as many data structures as you want.
- Some data structures that have proved useful over the years are:
  - Arrays
  - Linked Lists
  - Stacks
  - Queues
  - Trees

Data Structures and Algorithms

Role of Data Structures (Contd.)

- Use of an appropriate data structure, helps improve the efficiency of a program.
- The use of appropriate data structures also allows you to overcome some other programming challenges, such as:
  - Simplifying complex problems
  - Creating standard and reusable code components
  - Creating programs that are easy to understand and maintain

Data Structures and Algorithms

Types of Data Structures

- Data structures can be classified under the following two categories:
  - Static:
    - These are data structures whose size is fixed at compile time and does not grow or shrink at run time.
    - An example of a static data structure is an array.
  - Dynamic:
    - These are data structures whose size is not fixed at compile time and that grow or shrink at run time.
    - An example of a dynamic data structure is a linked list.

Just a Minute

- An array is a _____ data structure.

Just a Minute (Contd.)

- Solution:
  - Static

Demo: Problem Solving Using Algorithms

- Problem Statement:
  - Write an algorithm to accept a number and check whether the given number is an Armstrong number. In addition, write a program to implement this algorithm. A number is called an Armstrong number if the sum of the cubes of its digits is same as the number.

Demo: Problem Solving Using Algorithms (Contd.)

- Solution:
  - The following algorithm depicts the logic to check whether a given number, `n`, is an Armstrong number:

```
1. Accept the value of n.
2. Set temp=n, arm=0, rem=0.
3. Repeat the steps, a, b, and c, until n>0.
    a. Set rem=n%10.
    b. Set arm=arm+(rem*rem*rem).
    c. Set n=n/10.
4. If temp=arm, then:
       Display "It is an Armstrong number."
5. Else:
       Display "It is not an Armstrong number."
```

Demo: Problem Solving Using Algorithms (Contd.)

- To implement this algorithm, you need to perform the following tasks:
  1. Write the code.
  2. Execute the code and verify the output.

Designing Algorithms and Measuring Their Efficiency

- As there is no systematic method for algorithm design, designing an algorithm for a given problem is a difficult intellectual exercise.
- Moreover, there may be more than one algorithm to solve a given problem.
- Therefore, writing an effective algorithm for a new problem requires both, creativity and insight.

Identifying Techniques for Designing Algorithms

- Two commonly used techniques for designing algorithms are:
  - Divide and conquer approach
  - Greedy approach

Identifying Techniques for Designing Algorithms (Contd.)

- Divide and conquer is a powerful approach for solving conceptually difficult problems.
- Divide and conquer approach requires you to find a way of:
  - Breaking the problem into sub problems.
  - Solving the trivial cases.
  - Combining the solutions to the sub problems to solve the original problem.

Let us view an animation to understand the concept of divide and conquer approach.

Identifying Techniques for Designing Algorithms (Contd.)

- Algorithms based on greedy approach are used for solving optimization problems.
- Some examples of optimization problems are:
  - Finding the shortest distance from an originating city to a set of destination cities, given the distances between the pairs of cities.
  - Finding the minimum number of currency notes required for an amount, where an arbitrary number of notes for each denomination are available.
  - Selecting items with a maximum value from a given set of items, where the total weight of the selected items cannot exceed a given value.

Let us view an animation to understand the concept of greedy approach.

Just a Minute

- The _____ technique involves selecting the best available option at each step.

Just a Minute (Contd.)

- Solution:
  - Greedy

Designing Algorithms Using Recursion

- Recursion:
  - Refers to the technique of defining a process in terms of itself.
  - Is used to solve complex programming problems that are repetitive in nature.
  - Is implemented in a program by using a recursive procedure or function.
  - Is useful in writing clear, short, and simple programs.

Just a Minute

- _____ is used to solve complex programming problems that are repetitive in nature.

Just a Minute (Contd.)

- Solution:
  - Recursion

Determining the Efficiency of an Algorithm

- Factors that affect the efficiency of a program include:
  - Speed of the machine
  - Compiler
  - Operating system
  - Programming language
  - Size of the Input

Determining the Efficiency of an Algorithm (Contd.)

- The efficiency of an algorithm can be computed by determining the amount of resources it consumes.
- The primary resources that an algorithm consumes are:
  - Time: The CPU time required to execute the algorithm.
  - Space: The amount of memory used by the algorithm for its execution.
- The lesser resources an algorithm consumes, the more efficient it is.

Determining the Efficiency of an Algorithm (Contd.)

- Time/Space Trade-off:
  - It refers to a situation where you can reduce the use of memory at the cost of slower program execution, or reduce the running time at the cost of increased memory usage.
  - Example is data storage in compressed/uncompressed form.
- Memory is extensible, but time is not. Therefore, time considerations generally override memory considerations.

Determining the Efficiency of an Algorithm (Contd.)

- To measure the time efficiency of an algorithm, you can:
  1. Write a program based on the algorithm.
  2. Execute it.
  3. Measure the time it takes to run.
- The execution time that you measure in this case would depend on a number of factors, such as:
  - Speed of the machine
  - Compiler
  - Operating system
  - Programming language
  - Input data
- However, to determine how efficiently an algorithm solves a given problem, you need to determine how the execution time is affected by the nature of the algorithm.

Determining the Efficiency of an Algorithm (Contd.)

- Consider the following algorithm:

```
1. Set I = 0.                          // 1 assignment
2. While (I < n):                      // n comparisons
    a. Set J = 0.                      // n assignments
    b. While (J < n):                  // n × n comparisons
        i. Display (a[I][J]).          // n × n writes
        ii. Increment J by 1.          // n × n increments
    c. Increment I by 1.               // n increments
```

- The number of comparisons in the preceding algorithm is $(n^2 + n)$. The total running time is directly proportional to n^2.

Determining the Efficiency of an Algorithm (Contd.)

- The execution time of an algorithm is directly proportional to the number of key comparisons involved in the algorithm.
- The rate at which the running time of an algorithm increases as a result of an increase in the volume of input data is called the order of growth of the algorithm.
- The order of growth of an algorithm is defined by using the big O notation.
- The big O notation has been accepted as a fundamental technique for describing the efficiency of an algorithm.

Determining the Efficiency of an Algorithm (Contd.)

- The following figure depicts different orders of growth and their corresponding big O notations.

Order of Growth	Big O Notation
Constant	O (1)
Logarithmic	O (log n)
Linear	O (n)
Loglinear	O (n log n)
Quadratic	O (n^2)
Cubic	O (n^3)
Exponential	O (2^n), O (10^n)

Determining the Efficiency of an Algorithm (Contd.)

- According to their orders of growth, the big O notations can be arranged in an increasing order as:
  $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(10^n)$
- The following document depicts the orders of growth for the preceding big O notations.

The Orders of Growth

Demo: Designing Algorithms and Measuring Their Efficiency

- Problem Statement:
  - Write an algorithm to print the sum of the squares of first `n` numbers, where `n` is a number greater than or equal to 1. Also, find the efficiency of the algorithm.

Demo: Designing Algorithms and Measuring Their Efficiency (Contd.)

- Solution:
  - The following algorithm depicts the sum of the squares of first `n` numbers:

```
1. Accept the value of n.
2. Set i=1, sum=0.          // 2 assignments
3. Set sum=sum + i * i.     // 2n arithmetic operations
                            // n assignment operations
4. Increment i by 1.        // n increments
5. If i>n, then:            // n comparisons
       Go to step 7.
  Else:
       Go to step 3.
6. Display sum.             // 1 write
7. Exit.
```

  - The efficiency of the algorithm can be expressed as `O(n)`.

Summary

- In this session, you learned that:
  - An algorithm can be defined as a step-by-step procedure for solving a problem.
  - An algorithm has five important properties:
    - Finiteness
    - Definiteness
    - Input
    - Output
    - Effectiveness
  - An algorithm that provides maximum efficiency should be used for solving a problem.

Data Structures and Algorithms

Summary (Contd.)

- Data structures can be classified into the following two categories:
  - Static
  - Dynamic
- Two commonly used techniques for designing algorithms are:
  - Divide and conquer approach
  - Greedy approach
- Recursion refers to a technique of defining a process in terms of itself.
- The primary resources that an algorithm consumes are time and space.

Summary (Contd.)

- Time/space trade-off refers to a situation where you can:
  - Reduce the use of memory at the cost of slower program execution.
  - Reduce the running time at the cost of increased memory usage.
- The total running time of an algorithm is directly proportional to the number of comparisons involved in the algorithm.
- The order of growth of an algorithm is defined by using the big O notation.